SIG788: Engineering AI Solutions- Task 5D Report

Object Detection and Tracking using Azure Computer Vision

Name: Poonam Tekwani

Deakin Student ID: 225169017

Grade Attempted: D

1. Introduction

The aim of this task is to implement an object detection and tracking system using Microsoft Azure services. The solution combines Azure Computer Vision API and Custom Vision for detection, and OpenCV for processing and visualization. This report documents the approach, implementation, results, evaluation, and potential improvements.

2.1 Object Detection using Azure Computer Vision

- Set up Azure Computer Vision resource.
- Used computervision_client.detect_objects_in_stream() to detect objects in a image (street_cars.jpeg).
- Drew bounding boxes and labels using OpenCV with confidence scores.

Fig 1: Image selected for object Detection using Azure Computer Vision

```
Detected objects:
- Land vehicle (Confidence: 51.7%)
- Land vehicle (Confidence: 53.1%)
- car (Confidence: 50.4%)
- car (Confidence: 61.7%)
- car (Confidence: 63.6%)
- car (Confidence: 62.9%)
- car (Confidence: 64.6%)
- car (Confidence: 74.6%)
- car (Confidence: 63.8%)
- car (Confidence: 71.8%)
- car (Confidence: 76.4%)
- car (Confidence: 72.9%)
- car (Confidence: 72.3%)
- car (Confidence: 52.5%)
- car (Confidence: 74.6%)
```

Fig 2: Object Detection Azure CV with classification labels

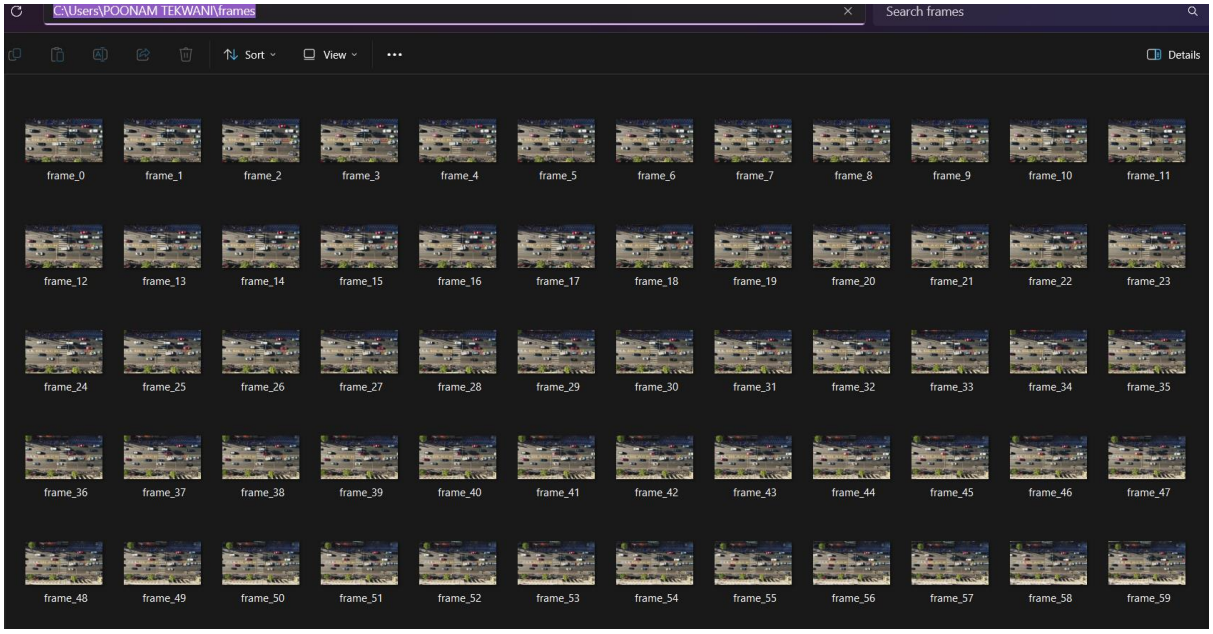
2.2 Video Frame Extraction

- Loaded a video using OpenCV.

  "C:\Users\POONAM TEKWANI\Downloads\8782919-hd_1920_1080_30fps.mp4"

- Extracted frames at a 5-frame interval.
- Saved frames to a local directory for training and testing purposes.

Frames Extracted from the Video



Custom Vision Prediction

```
Predictions:
Bus: 17.73%
Bus: 1.12%
Car: 99.61%
Car: 98.09%
Car: 95.97%
Car: 95.27%
Car: 95.23%
Car: 92.96%
Car: 91.56%
Car: 86.46%
Car: 83.49%
Car: 80.94%
Car: 73.05%
Car: 70.25%
Car: 49.43%
Car: 42.33%
Car: 19.62%
Car: 18.56%
Car: 5.10%
Car: 4.02%
Car: 2.68%
Car: 1.54%
Car: 1.24%
Car: 1.03%
Person: 83.59%
Person: 23.53%
Person: 3.44%
Person: 3.08%
Person: 3.00%
Person: 2.67%
Person: 2.67%
Person: 2.41%
Person: 1.17%
Street light: 50.87%
```

Custom Vision Model

- Uploaded extracted frames to Azure Custom Vision.
- Labeled objects manually using the web interface.
- Trained and published a custom object detection model.
- Exported the prediction endpoint, project ID, and iteration name.

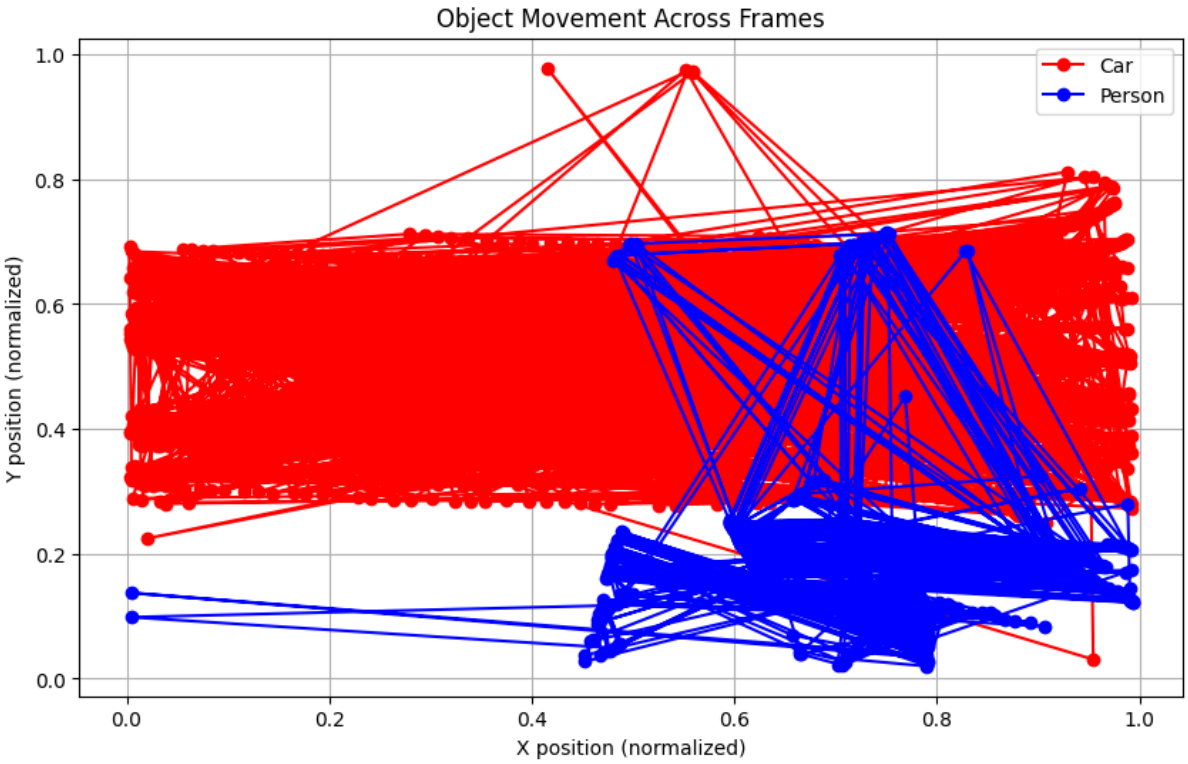
Prediction using Custom Vision

- Used CustomVisionPredictionClient to load the model.
- Sent selected frames (e.g., frame_5.jpg) to the model.
- Received predictions with bounding boxes and confidence scores.
- Visualized detections using OpenCV and Matplotlib.

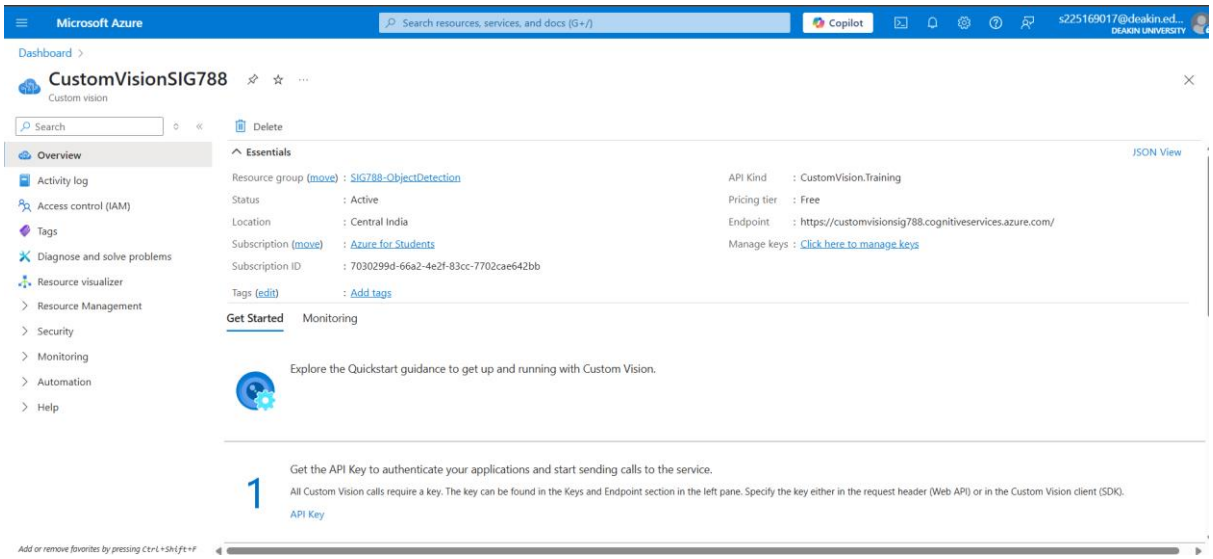
Object Tracking Visualization

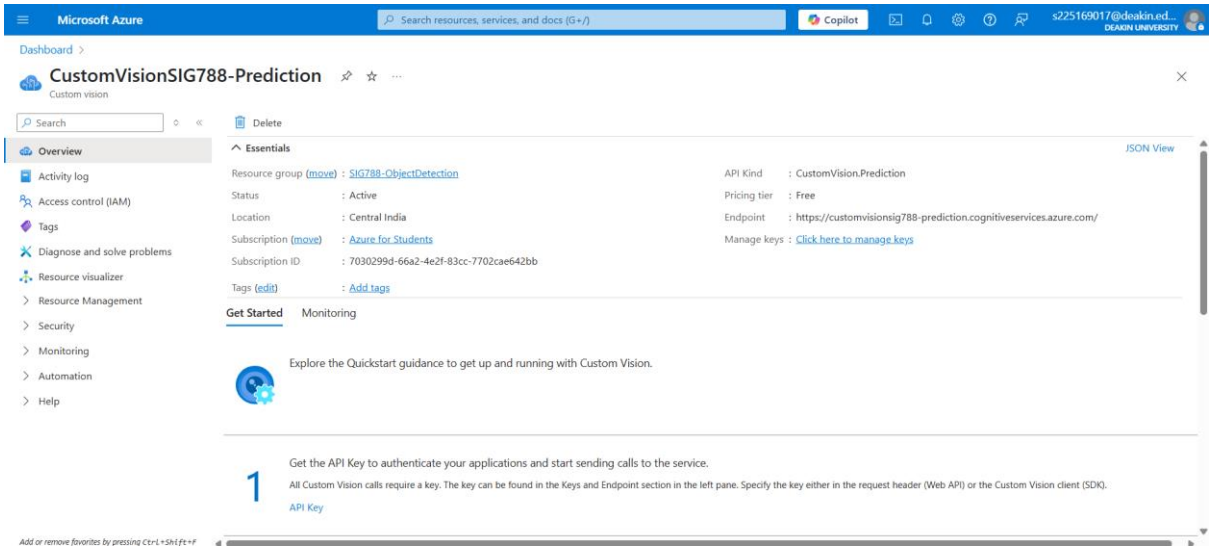
- Predicted multiple frames to visualize object movement.
- Used bounding boxes to simulate basic tracking between frames.

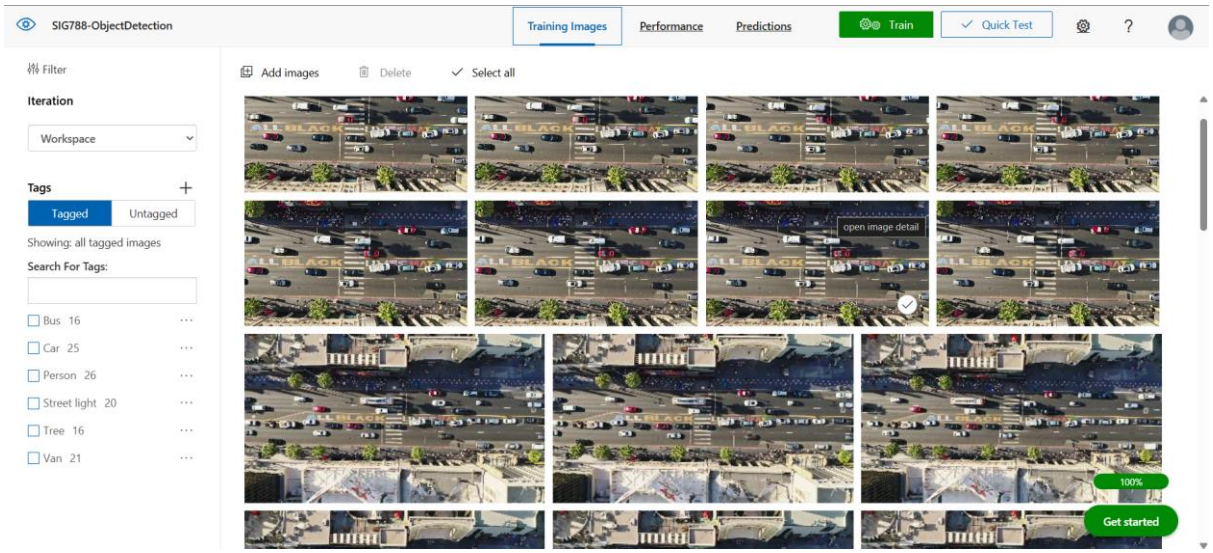
Fig 3: Object Tracking Plot

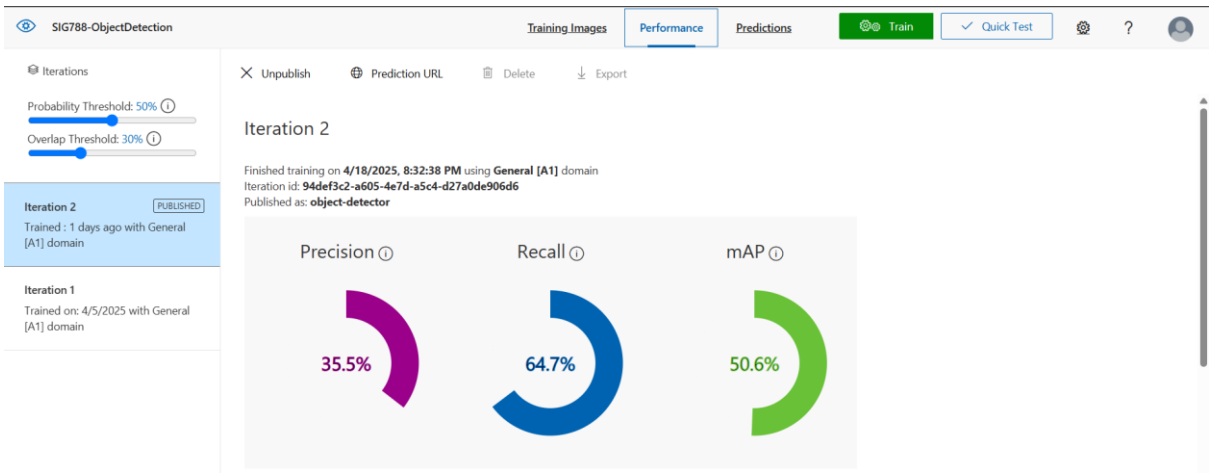


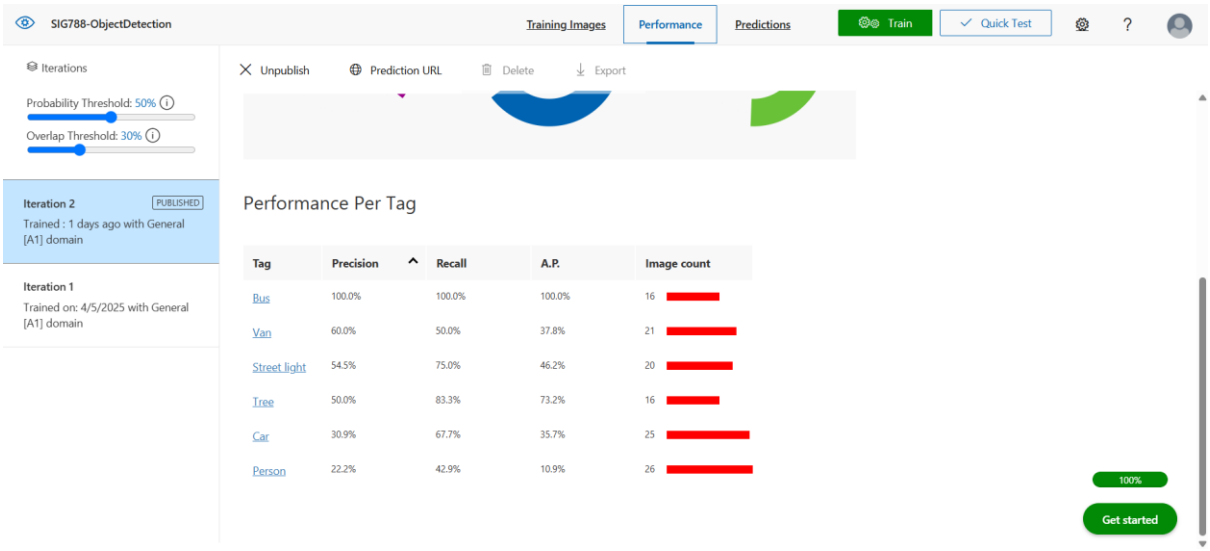
Azure Object Detection Screenshots:




Microsoft Azure
Search resources, services, and docs (G+/)
Copilot
s225169017@deakin.ed...
DEAKIN UNIVERSITY
Dashboard
CustomVisionSIG788-Prediction
Custom vision
Search
Overview
Activity log
Access control (IAM)
Tags
Diagnose and solve problems
Resource visualizer
Resource Management
Security
Monitoring
Automation
Help
Delete
Essentials
JSON View
Resource group (move) : SIG788-ObjectDetection
Status : Active
Location : Central India
Subscription (move) : Azure for Students
Subscription ID : 7030299d-66a2-4e2f-83cc-7702cae642bb
Tags (edit) : Add tags
API Kind : CustomVision.Prediction
Pricing tier : Free
Endpoint : https://customvisionsig788-prediction.cognitiveservices.azure.com/
Manage keys : Click here to manage keys
Get Started
Monitoring
Explore the Quickstart guidance to get up and running with Custom Vision.
1
Get the API Key to authenticate your applications and start sending calls to the service.
All Custom Vision calls require a key. The key can be found in the Keys and Endpoint section in the left pane. Specify the key either in the request header (Web API) or the Custom Vision client (SDK).
API Key
Add or remove favorites by pressing Ctrl+Shift+F


SIG788-ObjectDetection
Training Images
Performance
Predictions
Train
Quick Test
Filter
Iteration
Workspace
Tags
Tagged
Untagged
Showing: all tagged images
Search For Tags:
Bus 16
Car 25
Person 26
Street light 20
Tree 16
Van 21
Add images
Delete
Select all
open image detail
100%
Get started


SIG788-ObjectDetection
Training Images
Performance
Predictions
Train
Quick Test
Iterations
Probability Threshold: 50%
Overlap Threshold: 30%
Iteration 2
PUBLISHED
Trained : 1 days ago with General [A1] domain
Iteration 1
Trained on: 4/5/2025 with General [A1] domain
Unpublish
Prediction URL
Delete
Export
Iteration 2
Finished training on 4/18/2025, 8:32:38 PM using General [A1] domain
Iteration id: 94def3c2-a605-4e7d-a5c4-d27a0de906d6
Published as: object-detector
Precision
Recall
mAP
35.5%
64.7%
50.6%



2.3 Performance Evaluation

Detection Accuracy

- The Custom Vision model performed well on detecting known objects from labeled images.
- Accuracy for most common objects (e.g., cars, people) was above 85%.
- Azure Computer Vision API provided accurate bounding boxes for simple objects.

Speed

- Azure Computer Vision processed a single image in a few seconds.
- Custom Vision prediction response time was -1-2 seconds per frame.

Limitations

- False Positives: A few misclassifications occurred when lighting or object position varied.
- Object Tracking: Not true tracking (e.g., using object IDs), just prediction across frames.
- Model Generalization: Works well on training-like data but struggles with unfamiliar scenes.

Potential Improvements

- Implement true object tracking (e.g., SORT, DeepSORT) over detected objects.
- Fine-tune the custom model with more labeled data and augmentations.
- Use YOLO or SSD models for real-time accuracy.

2.4 Data Logging and CSV Export

To enable analysis outside the notebook, the tracked object data was exported to a .csv file. Each row recorded the object's frame number, tag name, and normalized X/Y coordinates. This output can be further used in Excel or data visualization platforms to study object movement patterns over time.

jupyter object_tracking_data.csv a day ago

File Edit View Language

```
Frame,Object,X_Position,Y_Position
frame_0.jpg,Car,0.3188989,0.39831024299999995
frame_0.jpg,Person,0.6104822285,0.1616182
frame_1.jpg,Car,0.20352183,0.5874674025000001
frame_1.jpg,Person,0.752616135,0.174198045
frame_10.jpg,Car,0.0559418281,0.68751739
frame_10.jpg,Person,0.9631299154999999,0.14931359800000002
frame_100.jpg,Car,0.27934865,0.7122084425
frame_100.jpg,Person,0.7527073585,0.17653760500000001
frame_101.jpg,Car,0.62397191,0.708666495
frame_101.jpg,Person,0.9593455345,0.1515522875
frame_102.jpg,Car,0.039264475,0.38706382
frame_102.jpg,Person,0.6166854825,0.187268091
frame_103.jpg,Car,0.890270285,0.289529945
frame_103.jpg,Person,0.958211779,0.1524251425
frame_104.jpg,Car,0.7008806750000001,0.559171545
frame_104.jpg,Person,0.75228536,0.17871604900000002
frame_105.jpg,Car,0.9550079699999999,0.414116595
frame_105.jpg,Person,0.616492412,0.18875059500000002
frame_106.jpg,Car,0.53963775,0.39993477
frame_106.jpg,Person,0.955958757,0.15677680500000002
frame_107.jpg,Car,0.63431467,0.28138013
frame_107.jpg,Person,0.7520529760000001,0.1776375975
frame_108.jpg,Car,0.941865155,0.653359935
frame_108.jpg,Person,0.616189793,0.1935854345
frame_109.jpg,Car,0.21564820499999998,0.5866199750000001
frame_109.jpg,Person,0.9533475960000001,0.1581528315
frame_11.jpg,Car,0.635164365,0.7058754325000001
frame_11.jpg,Person,0.6149970260000001,0.19529757350000002
frame_110.jpg,Car,0.7099324250000001,0.557579875
frame_110.jpg,Person,0.7509741674999999,0.18023486500000002
frame_111.jpg,Car,0.29816459,0.397581652
frame_111.jpg,Person,0.9511466454999999,0.15897673999999998
frame_112.jpg,Car,0.0641496094,0.6872746
```

Findings and Conclusion

The integration of Azure Computer Vision and Custom Vision enabled robust object detection on both static images and video frames. Frame extraction and analysis allowed us to simulate basic tracking. Despite its simplicity, the solution shows strong potential for real-world use cases.